

CODOMAX DIGITAL SOLUTIONS

MODULE 3 PROJECT REPORT

Cloud Services & Web Application Deployment

Cloud-Based Personal Portfolio Web Application

Submitted by: Sikandar Shah

Organization	Codomax Digital Solutions
Technology Stack	AWS
	Python
	Flask
	PostgreSQL
	Linux
	Nginx
Year	2026

Professional Internship Project Report

Document Control

Document Title	Module 3 — Cloud Services & Web Application Deployment
Project	Cloud-Based Personal Portfolio Web Application
Prepared By	Sikandar Shah
Organization	Codomax Digital Solutions
Status	Project Report — Evidence placeholders included

Table of Contents

1. Executive Summary
 2. Project Objectives
 3. Scope of Work
 4. Solution Architecture
 5. AWS Region and VPC Networking
 6. Security Groups
 7. EC2 Compute Environment
 8. Software Environment
 9. Amazon S3
 10. Amazon RDS PostgreSQL
 11. IAM Role and Permissions
 12. AWS Secrets Manager
 13. Database Initialization
 14. Application Health Testing
 15. S3 Application Integration
 16. Production Service with Unicorn
 17. Nginx Reverse Proxy
 18. Live Web Application
 19. Contact Form and Database Testing
 20. Application Logging
 21. CloudWatch Monitoring
 22. AWS Service Quotas
 23. Security Controls
 24. Challenges and Solutions
 25. Learning Outcomes
 26. Final Verification
 27. Conclusion
 28. Submission Deliverables
- Appendix A — Screenshot Evidence Index

1. Executive Summary

This project was completed as part of Module 3 of the Codomax Digital Solutions technical internship. The module focuses on practical cloud services and web application deployment, including compute, object storage, managed databases, virtual networking, IAM policies, secrets, environment variables, application logging, monitoring and resource quotas.

A personal portfolio web application was selected as the practical use case. The application presents the student's professional profile, skills, certifications, projects, experience and contact functionality. The cloud solution uses Amazon EC2 for application compute, Amazon RDS PostgreSQL for managed relational data, Amazon S3 for object storage, Amazon VPC for network isolation, Security Groups for traffic control, IAM for authorization, AWS Secrets Manager for sensitive database configuration and CloudWatch for monitoring and logging.

The project demonstrates how application code can be connected to cloud infrastructure through the flow: Application Code → Cloud Compute → Database/Storage → Networking → Live Web Application.

2. Project Objectives

The primary objective was to gain hands-on experience in deploying a functional application on AWS while applying basic cloud security principles.

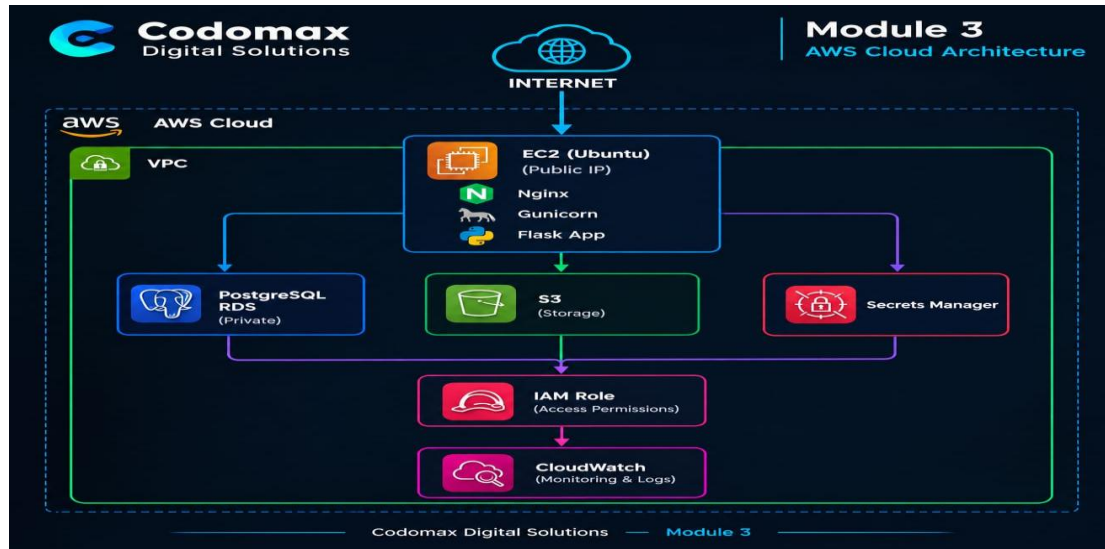
3. Scope of Work

The implementation scope covers AWS networking, EC2 compute, S3 object storage, RDS PostgreSQL, IAM, Secrets Manager, environment variables, Python/Flask development, database initialization, application health checks, S3 integration, production service configuration, Nginx, application logging, CloudWatch and Service Quotas.

4. Solution Architecture

The planned architecture uses a public EC2 application server and a private managed PostgreSQL database. Nginx receives web requests and forwards them to Gunicorn, which runs the Flask application. Flask communicates with RDS for structured data and S3 for object storage. Secrets Manager supplies database configuration securely through the EC2 IAM role.

Architecture Flow:



CloudWatch provides monitoring and log visibility for the environment.

5. AWS Region and VPC Networking

The project uses the AWS Asia Pacific (Mumbai) region, identified as ap-south-1. A dedicated project VPC was used to isolate the Module 3 environment. The intended network design contains public subnets for web-facing resources and private subnets for internal resources such as RDS.

The RDS database is configured without public access. Application-to-database communication takes place through the VPC using controlled Security Group rules.

6. Security Groups

Two Security Groups were planned for separation of responsibilities. The EC2 Security Group permits SSH access from the administrator's IP and HTTP access for the web application. The RDS Security Group permits TCP 5432 only from the EC2 application Security Group.

This approach avoids opening PostgreSQL directly to the public internet and demonstrates Security Group-based least-access networking.

7. EC2 Compute Environment

An Ubuntu LTS EC2 instance named module3-web-server was used as the application compute environment. The server hosts the Python virtual environment, Flask application, Gunicorn service and Nginx reverse proxy.

SSH access was configured using the module3-key key pair. The private key must remain on the administrator's local machine and must never be committed to GitHub.

8. Software Environment

The application uses Python with an isolated virtual environment. The main dependencies are Flask for the web application, Boto3 for AWS service integration, psycopg2-binary for PostgreSQL connectivity, Gunicorn as the WSGI application server and python-dotenv for environment configuration.

9. Amazon S3

An S3 bucket was created in the same AWS region as the application. Public access blocking and default encryption were enabled. S3 is used as application storage rather than as a public static website host.

The portfolio application is designed to retrieve the CV from an S3 object through the application. The EC2 IAM role provides the required object-level access.

10. Amazon RDS PostgreSQL

Amazon RDS PostgreSQL provides the managed relational database layer. The database contains a portfolio database with tables for project information and contact form messages.

The database is configured for private access and communicates with the EC2 application through TCP port 5432. The database credentials are not hard-coded in the application source code.

11. IAM Role and Permissions

The EC2 instance uses the module3-ec2-role IAM role. The role provides temporary AWS credentials to the EC2 workload so that the application can call required AWS services without embedding long-term AWS access keys.

The intended permissions are limited to the Module 3 S3 bucket and the required Secrets Manager secret. This supports the principle of least privilege.

12. AWS Secrets Manager

The database connection configuration is stored in the module3/database secret. The application retrieves the secret using Boto3 and the EC2 IAM role.

Sensitive values such as database passwords are intentionally excluded from the source repository and report screenshots. Any credentials exposed during troubleshooting should be rotated before final submission.

13. Database Initialization

The `init_db.py` script creates the `projects` and `contact_messages` tables and inserts the portfolio project records. Successful execution confirms that the application server can retrieve the database secret, establish a PostgreSQL connection and execute SQL statements.

Expected evidence: Database initialized successfully.

14. Application Health Testing

The Flask application includes a `/health` endpoint. It checks the application itself, the RDS database connection and S3 availability.

The successful test previously returned `application: online, database: online and s3: online`. This is an important integration test because it validates the three major application dependencies together.

15. S3 Application Integration

The application uses Boto3 to access the S3 bucket through the EC2 IAM role. The final application design provides a `/download-cv` route that retrieves `Sikandar-Shah-CV.pdf` from S3 and returns it as a downloadable PDF response.

This screenshot should only be marked complete after the S3 object has been uploaded and the application route has been successfully tested.

16. Production Service with Gunicorn

For production-style execution, the Flask development server should be replaced by Gunicorn. A `systemd` service keeps the application running after logout and allows it to start automatically with the operating system.

The service should bind Gunicorn to `127.0.0.1:5000` while Nginx provides public HTTP access.

17. Nginx Reverse Proxy

Nginx acts as the public-facing reverse proxy. Requests received on port 80 are forwarded to the Flask/Gunicorn application on `127.0.0.1:5000`.

This separation provides a standard deployment pattern in which Nginx handles public HTTP traffic while Gunicorn handles the Python application process.

18. Live Web Application

The portfolio website contains Home, About, Skills, Certifications, Projects, Experience and Contact sections. The visual design uses a professional dark navy and cyan theme.

The live website evidence should show the deployed application through the EC2 public address or configured domain after Nginx and the application service are operational.

19. Contact Form and Database Testing

The Contact section contains a form that accepts the visitor's name, email and message. A successful submission is inserted into the `contact_messages` table in PostgreSQL.

The evidence should include the successful form submission and, where appropriate, the corresponding database record without exposing sensitive credentials.

20. Application Logging

Application logging supports troubleshooting and operational visibility. The application records useful events such as startup activity, database errors, S3 errors and contact form processing.

Logs should be reviewed locally and, where configured, forwarded or made available through CloudWatch.

21. CloudWatch Monitoring

CloudWatch provides monitoring and operational visibility. EC2 metrics such as CPU utilization can be reviewed to understand server resource usage. CloudWatch Logs can be used to centralize application or system logs.

The report should contain evidence of both metrics and logs once the configured monitoring path has been tested.

22. AWS Service Quotas

AWS Service Quotas were reviewed to understand the limits associated with services used by the project. Quotas are important when applications scale or when additional instances, storage or networking resources are required.

23. Security Controls

The project applies several basic cloud security controls:

- RDS is not publicly accessible.
- Database access is restricted by Security Groups.
- SSH is restricted to the administrator's IP.
- AWS permissions are provided through an EC2 IAM role.
- Secrets are stored in Secrets Manager.
- S3 public access is blocked.
- Environment variables are used for non-secret configuration.
- `.env` and `.pem` files are excluded through `.gitignore`.
- AWS access keys and passwords are not stored in the GitHub repository.

24. Challenges and Solutions

During implementation, an IAM AccessDenied error occurred while the application attempted to retrieve the Secrets Manager value. The issue was resolved by adding the required Secrets Manager permission to the EC2 role.

A Secrets Manager JSON-format issue also caused a JSONDecodeError. The secret was corrected to valid JSON containing the required connection fields.

The application initially lacked the required environment configuration. Creating the .env file with the AWS region, S3 bucket and secret name restored the application's AWS configuration.

A Flask port conflict was also encountered because port 5000 was already occupied. The existing process was identified before continuing application testing.

25. Learning Outcomes

The project strengthened practical understanding of AWS infrastructure and cloud application deployment. Key learning outcomes include:

- Creating and working with VPC networking and subnets.
- Designing Security Group rules.
- Deploying and accessing an Ubuntu EC2 instance.
- Connecting an application to RDS PostgreSQL.
- Using S3 through Boto3.
- Using IAM roles instead of embedded AWS credentials.
- Managing secrets with AWS Secrets Manager.
- Building a Python Flask application.
- Running Flask with Gunicorn.
- Using Nginx as a reverse proxy.
- Testing application dependencies through a health endpoint.
- Understanding application logging and CloudWatch monitoring.
- Reviewing AWS Service Quotas.

26. Final Verification

Final verification should confirm that the complete application path works from the public web layer to the database and storage layers. The final evidence should demonstrate the running service, Nginx response, live website, contact database operation, logs, CloudWatch metrics/logs, Service Quotas and the final architecture.

Only verified tests should be marked as completed in the final submission.

27. Conclusion

Module 3 demonstrates the practical deployment of a cloud-based portfolio application using AWS services and a Python web stack. The project combines compute, networking, storage, database, identity, secrets and monitoring into one integrated environment.

The work provides a practical foundation for continued development in AWS Cloud, Cloud Security, Linux administration, secure application deployment and infrastructure operations.

Screenshot Evidence Index

Figure 1: VPC configuration

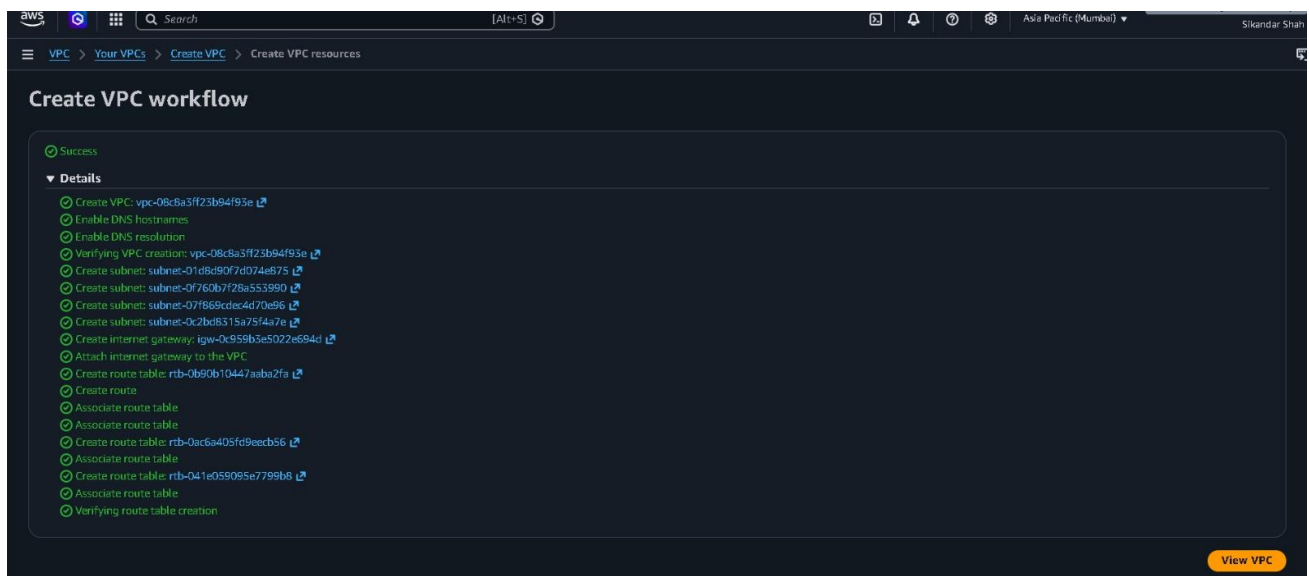


Figure 2: EC2 Security Group

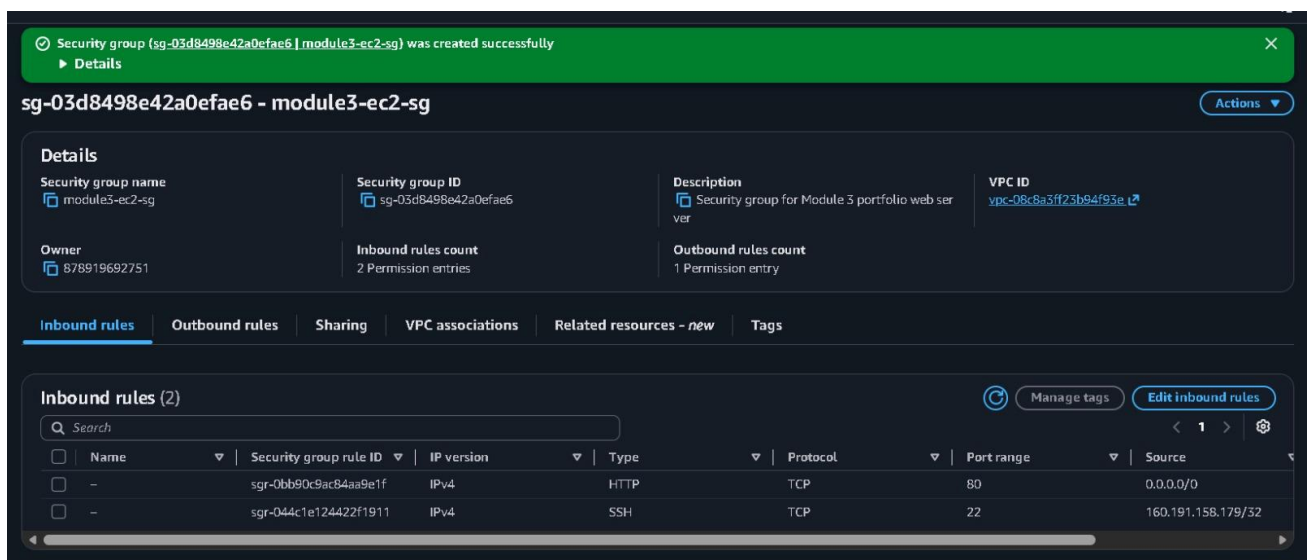


Figure 3: RDS Security Group

Security group (sg-05d13ed9f1050b593 | module3-rds-sg) was created successfully

sg-05d13ed9f1050b593 - module3-rds-sg

Details

Security group name module3-rds-sg	Security group ID sg-05d13ed9f1050b593	Description RDS Security group for Module 3 portfolio web server	VPC ID vpc-08c8a3ff23b94f93e
Owner 878919692751	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

Inbound rules (1)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-0b863eb1e7d661f43	-	PostgreSQL	TCP	5432	sg-03d8498e42a0efae6...

Figure 4: EC2 instance running

Successfully initiated stopping of i-09f627563aad3c278

Instances (3) Info

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
Lab2-env	i-09f627563aad3c278	Stopping	t3.micro	-	ap-south-1b	ec2-65-2-173-92.ap-sou...	65.2.173.92
Module2-Linux-Server	i-00c06d28a55acf54a	Stopped	t3.micro	-	ap-south-1a	-	-
module3-web-server	i-0793999ee3b2200bf	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-65-0-76-17.ap-sout...	65.0.76.17

Figure 5: SSH connection from local machine

```
PS C:\Users\HB Comp> cd "$HOME\Downloads"
PS C:\Users\HB Comp\Downloads> dir

Directory: C:\Users\HB Comp\Downloads

Mode                LastWriteTime         Length Name
----                -
-a-----          9/20/2026   2:34 AM           519583 Copilot_20260920_023424.png
-a-----          9/20/2026   2:56 AM           755328 Copilot_20260920_025605.png
-a-----          9/20/2026   3:46 AM        2886560 MBSetup-2.2.exe
-a-----          9/20/2026   3:15 AM        2364456 Module 1_ Cloud Computing Foundations.docx
-a-----          9/21/2026   1:26 AM           1674 module2-key.pem
-a-----          9/22/2026  12:51 AM           1678 module3-key.pem

PS C:\Users\HB Comp\Downloads> ssh -i ".\module3-key.pem" ubuntu@65.0.76.17
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 22 10:35:10 UTC 2026

System load:  0.0               Temperature:  -273.1 C
Usage of /:   30.9% of 6.61GB   Processes:    117
Memory usage: 27%              Users logged in: 0
Swap usage:  0%               IPv4 address for ens5: 10.0.3.129

 * Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-10-0-3-129:~$
```

Figure 6: S3 bucket configuration

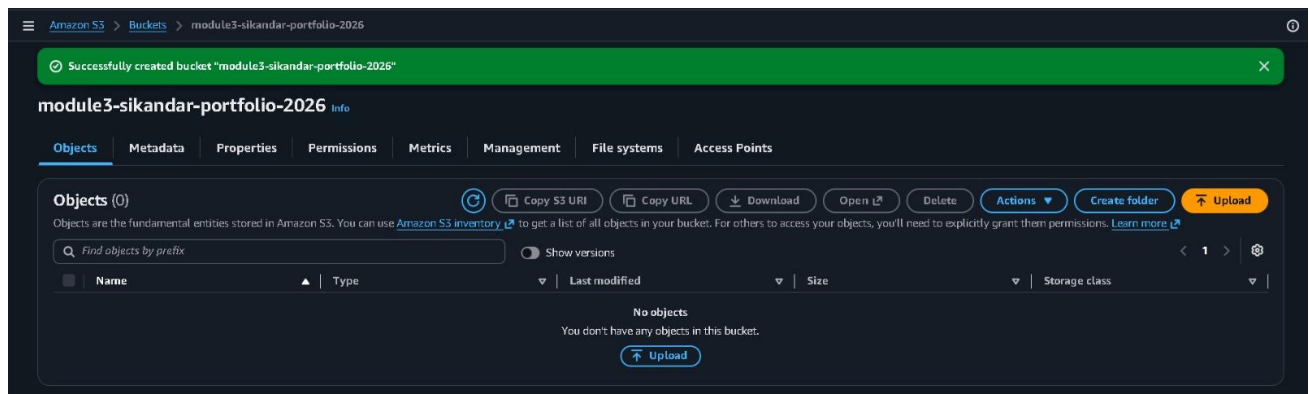
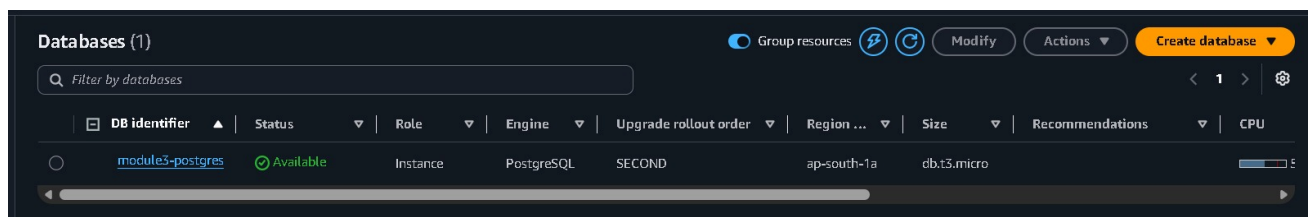


Figure 7: RDS instance available



EC2 IAM role

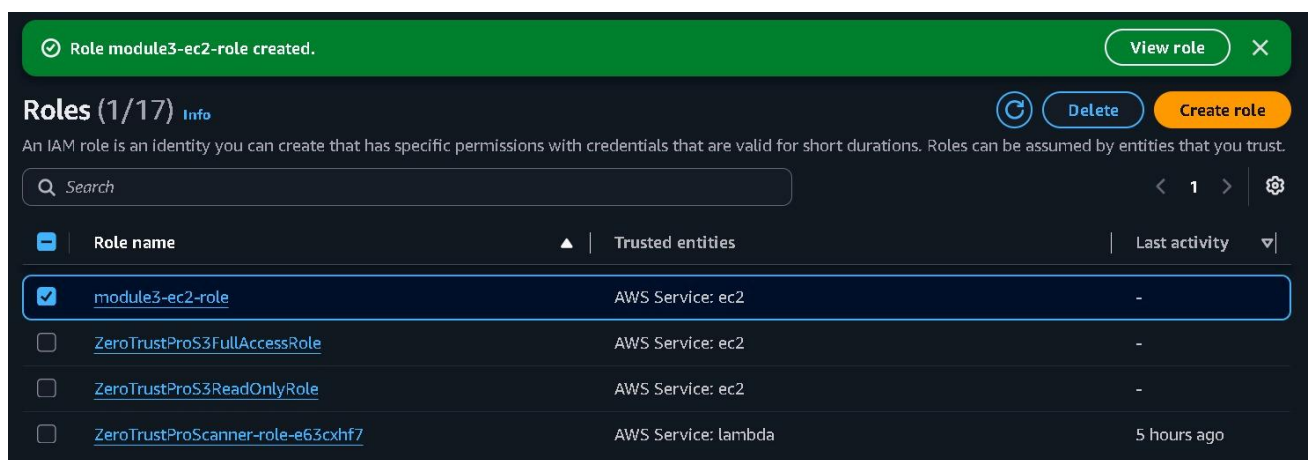


Figure 8: Successful RDS connection

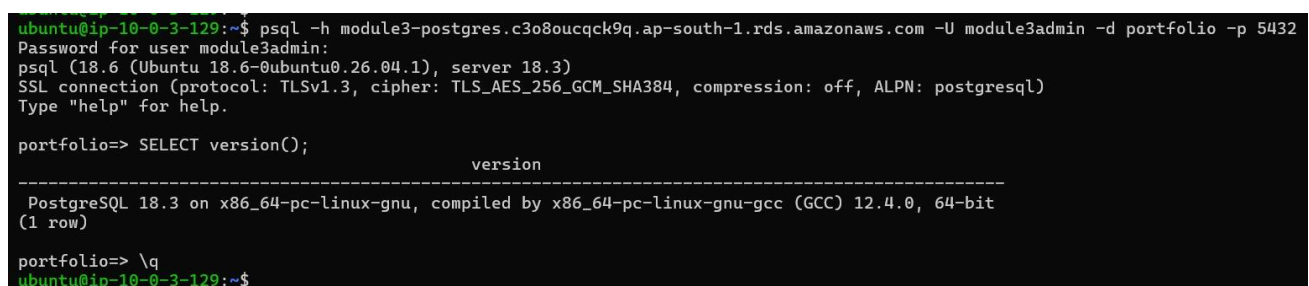


Figure 9: Secrets Manager secret

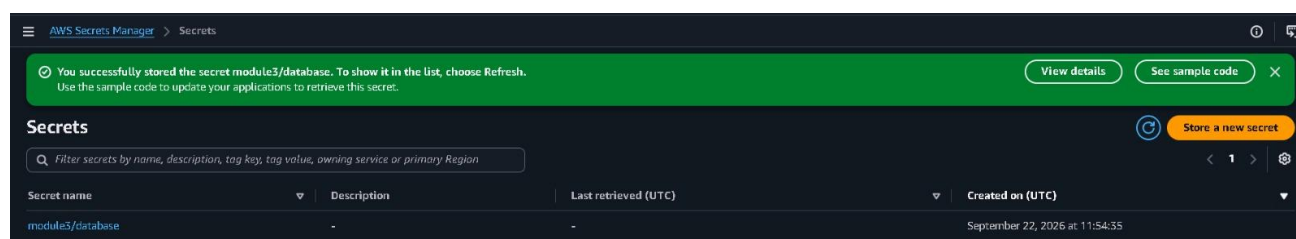


Figure 10: IAM access test

```
ubuntu@ip-10-0-3-129:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1006-aws source/x86_64.ubuntu.26
ubuntu@ip-10-0-3-129:~$ aws sts get-caller-identity
{
  "UserId": "ARO4ZI5SGHH5MCNRGQ55:i-0793999ee3b2200bf",
  "Account": "878919692751",
  "Arn": "arn:aws:sts::878919692751:assumed-role/module3-ec2-role/i-0793999ee3b2200bf"
}
ubuntu@ip-10-0-3-129:~$ aws s3 ls
2026-08-11 04:03:37 amazon-sagemaker-878919692751-ap-south-1-4yqcdco7xptc75
2026-08-10 07:20:44 aws-cloudtrail-logs-878919692751-460bd8da
2026-09-02 07:48:45 elasticbeanstalk-ap-south-1-878919692751
2026-09-22 10:45:34 module3-sikandar-portfolio-2026
2026-08-05 07:42:37 zerotrustpro-logs
2026-08-05 08:08:16 zerotrustpro-sikandar-site
ubuntu@ip-10-0-3-129:~$
```

Figure 11: Database initialization successful

```
ubuntu@ip-10-0-3-129:~$ cd ~/portfolio
ubuntu@ip-10-0-3-129:~/portfolio$ source venv/bin/activate
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ python3 init_db.py
2026-09-22 12:41:59,198 INFO Found credentials from IAM Role: module3-ec2-role
Database initialized successfully.
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ |
```

Figure 12: Application health test

```
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ cd ~/portfolio
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ source venv/bin/activate
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ curl http://127.0.0.1:5000/health
{"application":"online","database":"online","s3":"online"}
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ |
```

Figure 13: Application-level S3 integration

```
ubuntu@ip-10-0-3-129:~/portfolio$ source venv/bin/activate
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ python3 -c "import boto3, os; from dotenv import load_dotenv; load_dotenv(); s3=boto3.client('s3', region_name=os.getenv('AWS_REGION')); s3.upload_file('static/uploads/module3-test.txt', os.getenv('S3_BUCKET'), 'module3-test.txt'); print('S3 upload successful')"
S3 upload successful
(venv) ubuntu@ip-10-0-3-129:~/portfolio$
```

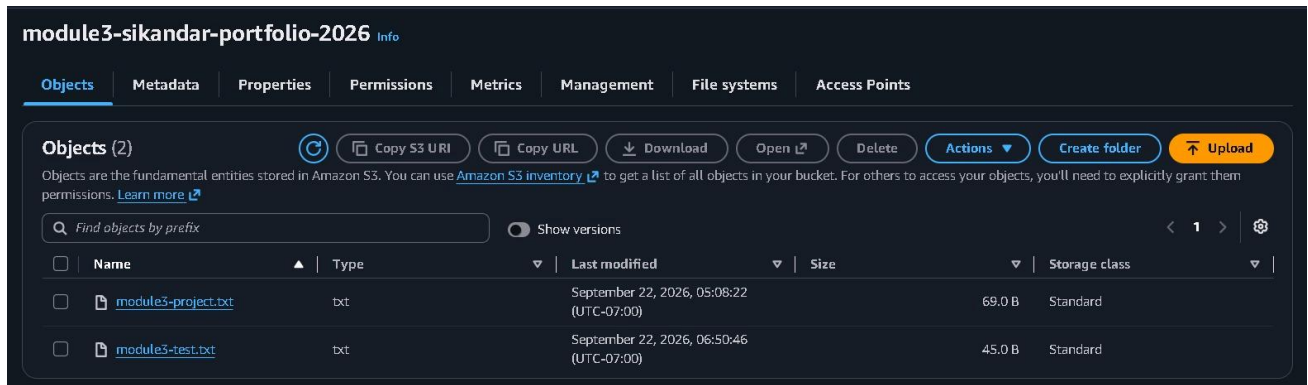


Figure 14: Gunicorn/systemd service running

```
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo systemctl start portfolio
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo systemctl status portfolio
● portfolio.service - Sikandar Shah Flask Portfolio
   Loaded: loaded (/etc/systemd/system/portfolio.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-22 14:04:28 UTC; 4s ago
  Invocation: eae504f45424457593823a8b54ed405f
    Main PID: 32811 (gunicorn)
      Tasks: 1 (limit: 627)
     Memory: 14.7M (peak: 14.7M)
        CPU: 105ms
   CGroup: /system.slice/portfolio.service
           └─32811 /home/ubuntu/portfolio/venv/bin/python3 /home/ubuntu/portfolio/venv/bin/gunicorn --workers 2 --bind 127.0.0.1:80
```

Figure 15: Nginx configuration/test

```
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo nano /etc/nginx/sites-available/portfolio
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo rm /etc/nginx/sites-enabled/default
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo ln -s /etc/nginx/sites-available/portfolio /etc/nginx/sites-enabled/portfolio
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo systemctl restart nginx
(venv) ubuntu@ip-10-0-3-129:~/portfolio$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-22 14:07:54 UTC; 9s ago
  Invocation: f54fcd09e45e4cca8058bf5d1ed0e9d5
     Docs: man:nginx(8)
  Process: 32991 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 32993 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Main PID: 32994 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3.2M (peak: 3.5M)
        CPU: 31ms
   CGroup: /system.slice/nginx.service
           └─32994 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─32995 "nginx: worker process"
               └─32996 "nginx: worker process"
```

Sep 22 14:07:53 ip-10-0-3-129 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...

Sep 22 14:07:54 ip-10-0-3-129 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.

```
(venv) ubuntu@ip-10-0-3-129:~/portfolio$
```

Figure 16: Live portfolio website



Figure 17: Contact form/database test

```
ubuntu@ip-10-0-3-129:~$ psql -h module3-postgres.c3o8oucck9q.ap-south-1.rds.amazonaws.com -U module3admin -d portfolio
Password for user module3admin:
psql (18.6 (Ubuntu 18.6-0ubuntu0.26.04.1), server 18.3)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

portfolio=> SELECT * FROM contact_messages;
 id | name | email | message | created_at
-----+-----+-----+-----+-----
  1 | Sikandar Shah | traders.shah10@gmail.com | Testing the Module 3 portfolio contact form. | 2026-09-22 14:13:44.322401
(1 row)

portfolio=> \q
ubuntu@ip-10-0-3-129:~$
```

Figure 18: Application logs

```
ubuntu@ip-10-0-3-129:~$ sudo journalctl -u portfolio --no-pager -n 50
Sep 22 14:47:46 ip-10-0-3-129 systemd[1]: portfolio.service: Consumed 1.447s CPU time over 27min 4.065s wall clock time, 109.1M memory peak.
Sep 22 14:47:46 ip-10-0-3-129 systemd[1]: Started portfolio.service - Sikandar Shah Flask Portfolio.
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:47:46 +0000] [34067] [INFO] Starting gunicorn 26.2.0
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:47:46 +0000] [34067] [INFO] Listening at: http://127.0.0.1:8000 (34067)
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:47:46 +0000] [34067] [INFO] Using worker: sync
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34072]: [2026-09-22 14:47:46 +0000] [34072] [INFO] Booting worker with pid: 34072
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34073]: [2026-09-22 14:47:46 +0000] [34073] [INFO] Booting worker with pid: 34073
Sep 22 14:47:46 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:47:46 +0000] [34067] [INFO] Control socket listening at /home/ubuntu/.gunicorn/gunicorn.ctl
Sep 22 14:47:47 ip-10-0-3-129 gunicorn[34072]: 2026-09-22 14:47:47.782 INFO Found credentials from IAM Role: module3-ec2-role
Sep 22 14:48:16 ip-10-0-3-129 gunicorn[34073]: 2026-09-22 14:48:16.293 INFO Found credentials from IAM Role: module3-ec2-role
Sep 22 14:53:32 ip-10-0-3-129 systemd[1]: /etc/systemd/system/portfolio.service:1: Assignment outside of section. Ignoring.
Sep 22 14:53:32 ip-10-0-3-129 systemd[1]: /etc/systemd/system/portfolio.service:2: Assignment outside of section. Ignoring.
Sep 22 14:53:32 ip-10-0-3-129 systemd[1]: /etc/systemd/system/portfolio.service:3: Assignment outside of section. Ignoring.
Sep 22 14:53:43 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:53:43 +0000] [34067] [INFO] Handling signal: term
Sep 22 14:53:43 ip-10-0-3-129 systemd[1]: Stopping portfolio.service...
Sep 22 14:53:43 ip-10-0-3-129 gunicorn[34073]: [2026-09-22 14:53:43 +0000] [34073] [INFO] Worker exiting (pid: 34073)
Sep 22 14:53:43 ip-10-0-3-129 gunicorn[34072]: [2026-09-22 14:53:43 +0000] [34072] [INFO] Worker exiting (pid: 34072)
Sep 22 14:53:43 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:53:43 +0000] [34067] [INFO] Worker (pid:34073) was sent SIGTERM!
Sep 22 14:53:43 ip-10-0-3-129 gunicorn[34067]: [2026-09-22 14:53:43 +0000] [34067] [INFO] Shutting down: Master
Sep 22 14:53:44 ip-10-0-3-129 systemd[1]: portfolio.service: Deactivated successfully.
Sep 22 14:53:44 ip-10-0-3-129 systemd[1]: Stopped portfolio.service.
Sep 22 14:53:44 ip-10-0-3-129 systemd[1]: portfolio.service: Consumed 1.228s CPU time over 5min 57.414s wall clock time, 108.8M memory peak.
Sep 22 14:53:44 ip-10-0-3-129 systemd[1]: Started portfolio.service.
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:53:44 +0000] [34282] [INFO] Starting gunicorn 26.2.0
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:53:44 +0000] [34282] [INFO] Listening at: http://127.0.0.1:8000 (34282)
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:53:44 +0000] [34282] [INFO] Using worker: sync
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34288]: [2026-09-22 14:53:44 +0000] [34288] [INFO] Booting worker with pid: 34288
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34289]: [2026-09-22 14:53:44 +0000] [34289] [INFO] Booting worker with pid: 34289
Sep 22 14:53:44 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:53:44 +0000] [34282] [INFO] Control socket listening at /home/ubuntu/.gunicorn/gunicorn.ctl
Sep 22 14:53:57 ip-10-0-3-129 gunicorn[34289]: 2026-09-22 14:53:57.113 INFO Found credentials from IAM Role: module3-ec2-role
Sep 22 14:58:15 ip-10-0-3-129 gunicorn[34289]: [2026-09-22 14:58:15 +0000] [34289] [INFO] Worker exiting (pid: 34289)
Sep 22 14:58:15 ip-10-0-3-129 systemd[1]: Stopping portfolio.service - Sikandar Shah Flask Portfolio...
Sep 22 14:58:15 ip-10-0-3-129 gunicorn[34288]: [2026-09-22 14:58:15 +0000] [34288] [INFO] Worker exiting (pid: 34288)
Sep 22 14:58:15 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:58:15 +0000] [34282] [INFO] Handling signal: term
Sep 22 14:58:15 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:58:15 +0000] [34282] [INFO] Worker (pid:34288) was sent SIGTERM!
Sep 22 14:58:15 ip-10-0-3-129 gunicorn[34282]: [2026-09-22 14:58:15 +0000] [34282] [INFO] Worker (pid:34289) was sent SIGTERM!
```

Figure 19: CloudWatch metrics

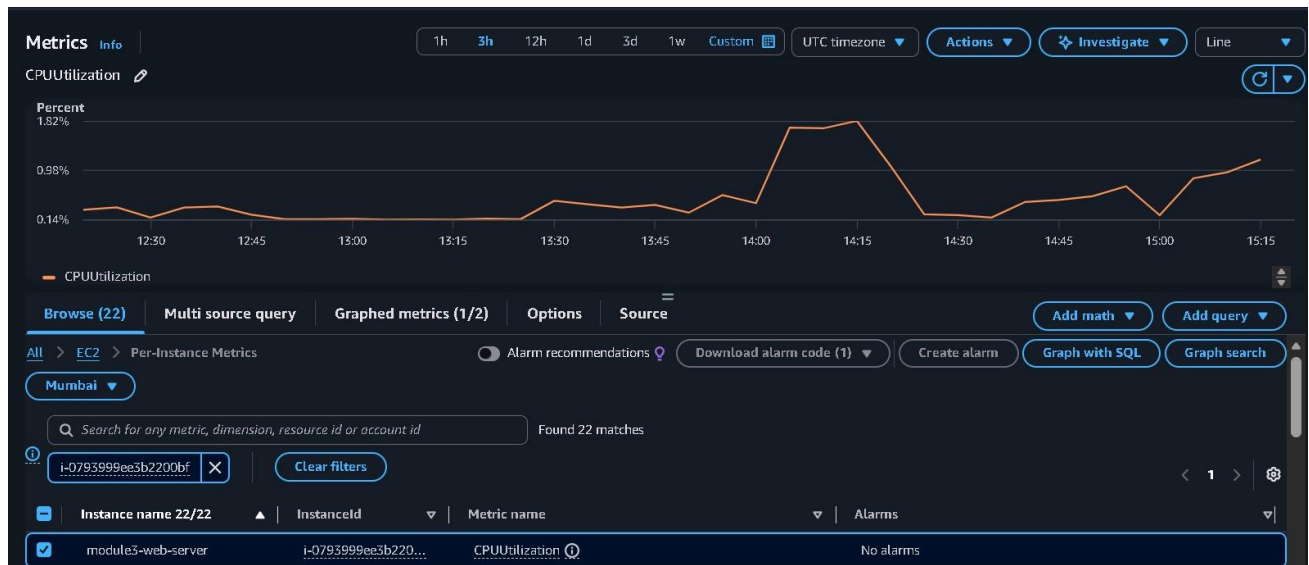


Figure 20: CloudWatch logs

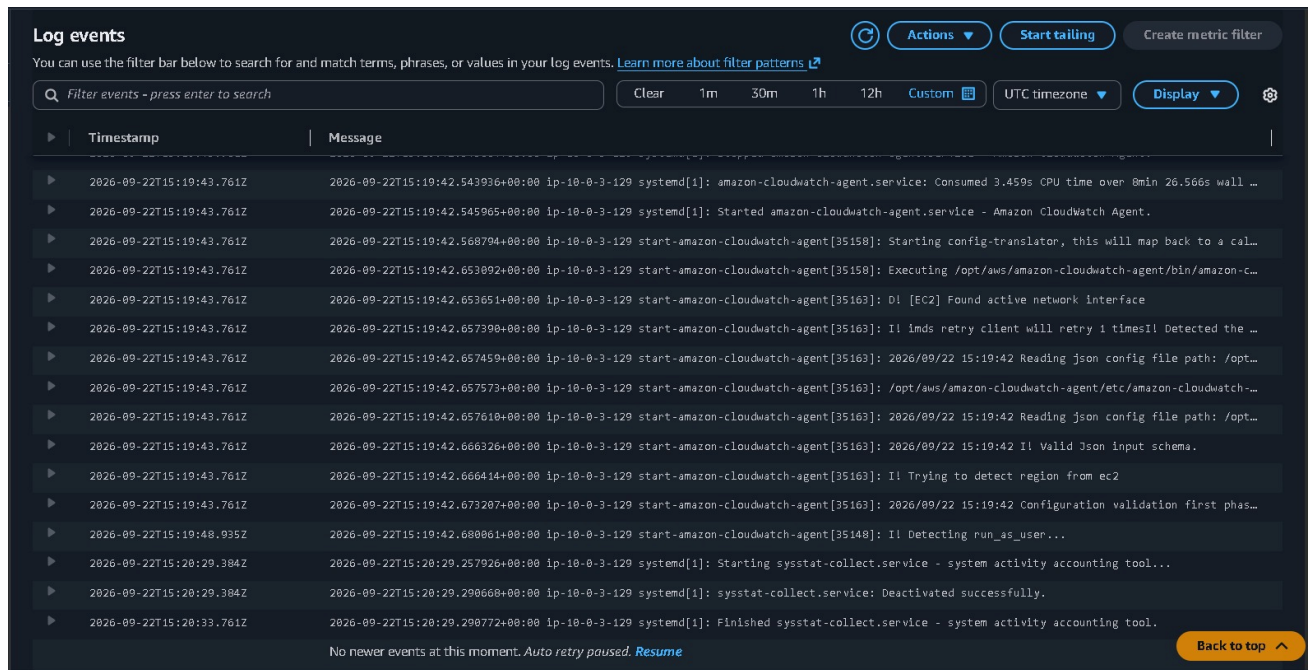
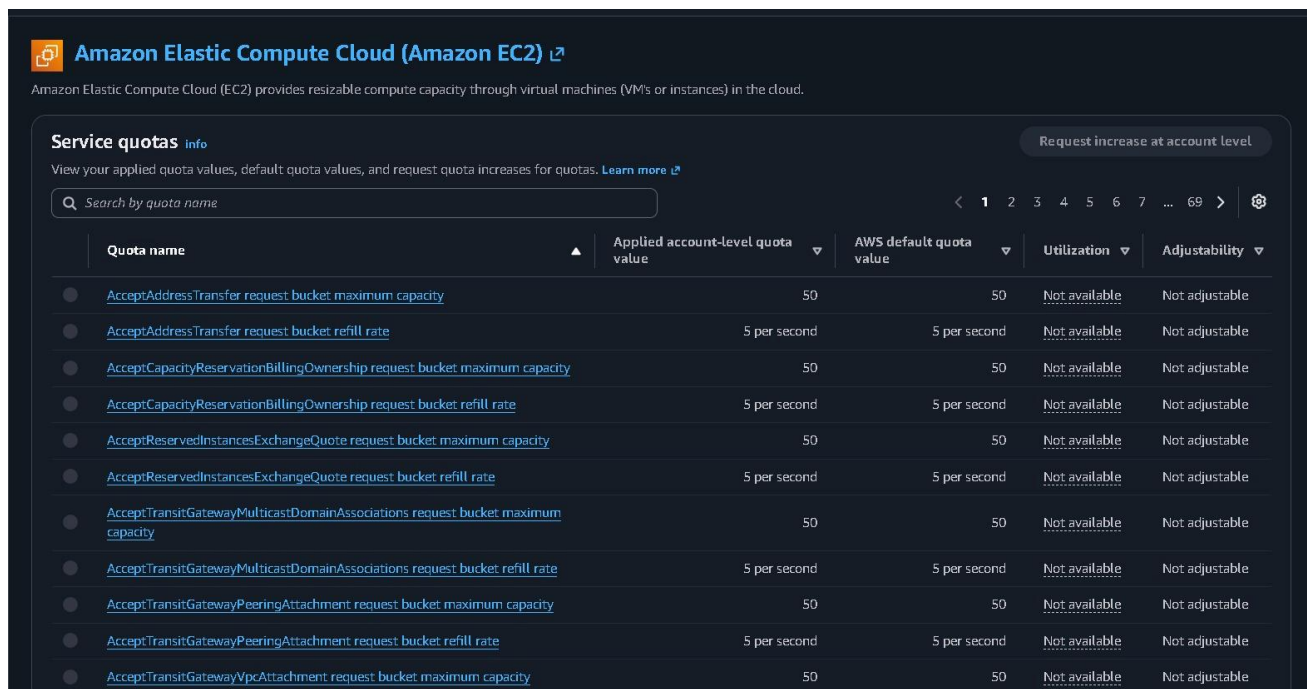


Figure 21: AWS Service Quotas



Amazon Elastic Compute Cloud (Amazon EC2)

Amazon Elastic Compute Cloud (EC2) provides resizable compute capacity through virtual machines (VMs or instances) in the cloud.

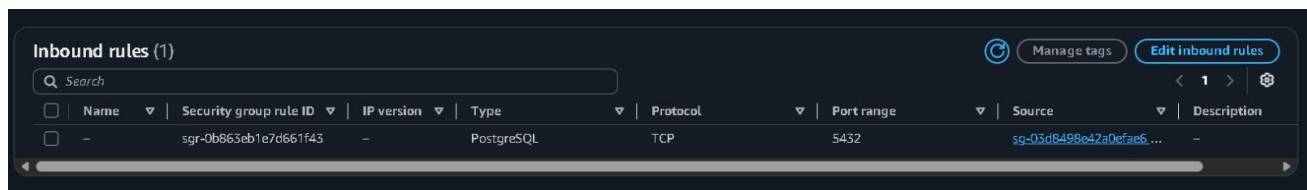
Service quotas info

View your applied quota values, default quota values, and request quota increases for quotas. [Learn more](#)

Search by quota name

Quota name	Applied account-level quota value	AWS default quota value	Utilization	Adjustability
AcceptAddressTransfer request bucket maximum capacity	50	50	Not available	Not adjustable
AcceptAddressTransfer request bucket refill rate	5 per second	5 per second	Not available	Not adjustable
AcceptCapacityReservationBillingOwnership request bucket maximum capacity	50	50	Not available	Not adjustable
AcceptCapacityReservationBillingOwnership request bucket refill rate	5 per second	5 per second	Not available	Not adjustable
AcceptReservedInstancesExchangeQuote request bucket maximum capacity	50	50	Not available	Not adjustable
AcceptReservedInstancesExchangeQuote request bucket refill rate	5 per second	5 per second	Not available	Not adjustable
AcceptTransitGatewayMulticastDomainAssociations request bucket maximum capacity	50	50	Not available	Not adjustable
AcceptTransitGatewayMulticastDomainAssociations request bucket refill rate	5 per second	5 per second	Not available	Not adjustable
AcceptTransitGatewayPeeringAttachment request bucket maximum capacity	50	50	Not available	Not adjustable
AcceptTransitGatewayPeeringAttachment request bucket refill rate	5 per second	5 per second	Not available	Not adjustable
AcceptTransitGatewayVpcAttachment request bucket maximum capacity	50	50	Not available	Not adjustable

Figure 22: Final architecture/integration verification



Inbound rules (1)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0b863eb1e7d661f43	-	PostgreSQL	TCP	5432	sg-03d8498e42a0efae6 ...	-

Final Security Checklist

- Do not commit .env files.
- Do not commit .pem private keys.
- Do not commit AWS access keys.
- Do not include database passwords in README, source code or screenshots.
- Rotate any database password that was exposed during troubleshooting before final submission.
- Keep RDS private and restrict PostgreSQL access to the EC2 application Security Group.
- Keep S3 Block Public Access enabled unless a documented application requirement says otherwise.
- Use the EC2 IAM role for AWS service access.
- Verify the final GitHub repository before publishing.